ESP32 Dual Axis Motion Control

Setup, web interface, Gen.Out outputs, and Program tab walkthrough

Revision: 2026-07-07 - Added Gen.Out1 / Gen.Out2 naming, Gen1 / Gen2 program commands, R1 / R2 / R3 program relay commands, and VX / VY speed commands.

Purpose: This document is a practical reference for wiring, configuring, and operating the ESP32 dual-axis stepper motor controller through the embedded web interface.

1. System Overview

The firmware controls two DRV8825 stepper drivers, two home switch inputs, two timed general outputs, three program relay outputs, and an embedded HTML interface over Wi-Fi. The interface contains Dashboard, Motor Settings, Program, and Wi-Fi Settings tabs.

Motor 1 is used as the X axis and Motor 2 is used as the Y axis. Gen.Out1 is controlled on GPIO23 and Gen.Out2 is controlled on GPIO5. The program relay outputs are R1 on GPIO2, R2 on GPIO12, and R3 on GPIO15.

2. Hardware Connections

The ESP32, DRV8825 drivers, relay modules, and control-side power supply must share a common ground. Motor power and ESP32 power may be supplied separately, but their reference ground must be common.

Function	ESP32 GPIO	Connection
X / Motor 1 STEP	GPIO25	DRV8825 Motor 1 STEP
X / Motor 1 DIR	GPIO26	DRV8825 Motor 1 DIR
X / Motor 1 ENABLE	GPIO27	DRV8825 Motor 1 ENABLE
X / Motor 1 HOME	GPIO32	Home switch signal, other switch terminal to GND
X / Motor 1 M0/M1/M2	GPIO14/13/4	DRV8825 M0/M1/M2
Y / Motor 2 STEP	GPIO18	DRV8825 Motor 2 STEP
Y / Motor 2 DIR	GPIO19	DRV8825 Motor 2 DIR
Y / Motor 2 ENABLE	GPIO21	DRV8825 Motor 2 ENABLE
Y / Motor 2 HOME	GPIO33	Home switch signal, other switch terminal to GND
Y / Motor 2 M0/M1/M2	GPIO16/17/22	DRV8825 M0/M1/M2
Gen.Out1	GPIO23	Relay module input IN1 or transistor driver input
Gen.Out2	GPIO5	Relay module input IN2 or transistor driver input
R1 program relay	GPIO2	Relay module input or transistor driver input
R2 program relay	GPIO12	Relay module input or transistor driver input
R3 program relay	GPIO15	Relay module input or transistor driver input

GPIO5, GPIO2, GPIO12, and GPIO15 are ESP32 boot strapping pins. Relay inputs connected to these pins must not pull the pins to an invalid level during reset. If boot problems occur, move the related pin definition in `main/Config.h` to another safe free output pin.

3. Wi-Fi And Interface Access

On each boot, the ESP32 starts its own access point. If station credentials are saved, it also attempts to connect to the local network at the same time.

Field	Value
AP SSID	ESP32-MOTOR
AP Password	12345678
AP URL	http://192.168.4.1
STA URL	ESP32 IP address assigned by the router

4. Dashboard Tab

The Dashboard tab shows live telemetry and manual controls. For the X and Y axes, it displays current position, target position, remaining distance, speed, acceleration, home switch state, and driver enable state.

Move To Position fields move an axis to an absolute target position.

Jog buttons move the selected axis by -10 or +10 relative units.

Home starts the two-stage homing routine.

Set Zero stores the current position as zero.

Enable Driver and Disable Driver manually control driver power.

All Stop stops both motors, stops the running program, and turns off the Gen.Out outputs.

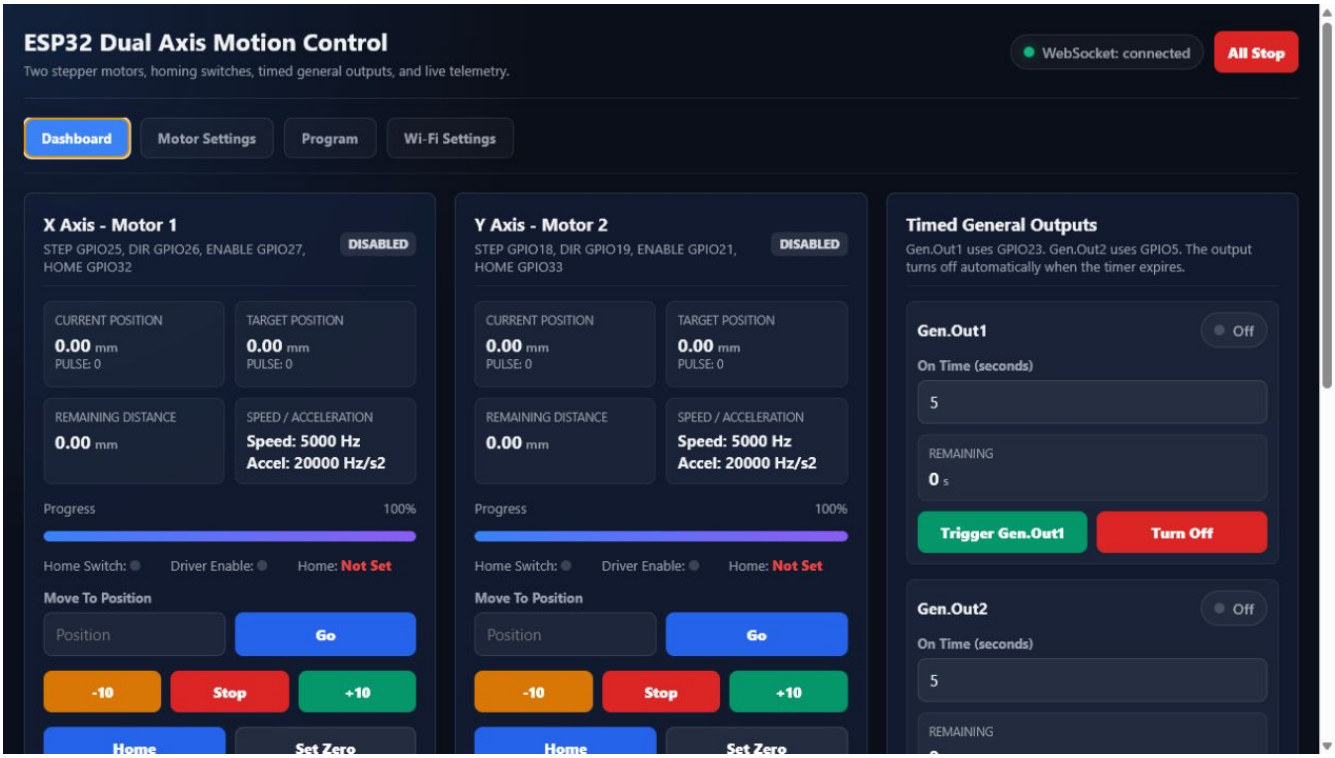
5. Timed General Outputs

The Timed General Outputs panel on the Dashboard can trigger Gen.Out1 and Gen.Out2 manually for a selected duration. The entered duration is in seconds. When the countdown expires, the firmware turns the output off automatically. The Turn Off button cancels the timer and turns the output off immediately.

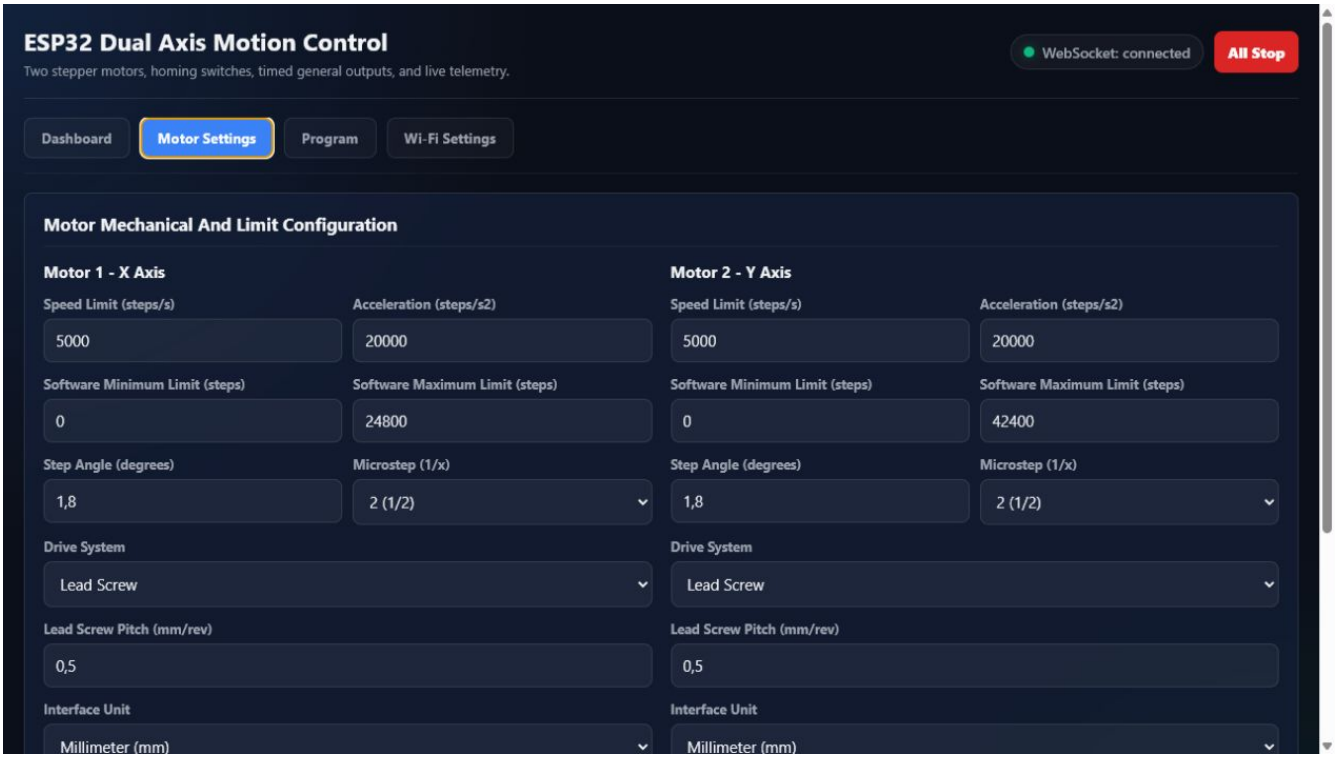
The Gen1 / Gen2 and R1 / R2 / R3 commands in the Program tab are direct ON / OFF commands, not timed commands. An output that is turned on inside a program remains on until a later line turns it off.

6. Interface Screenshots

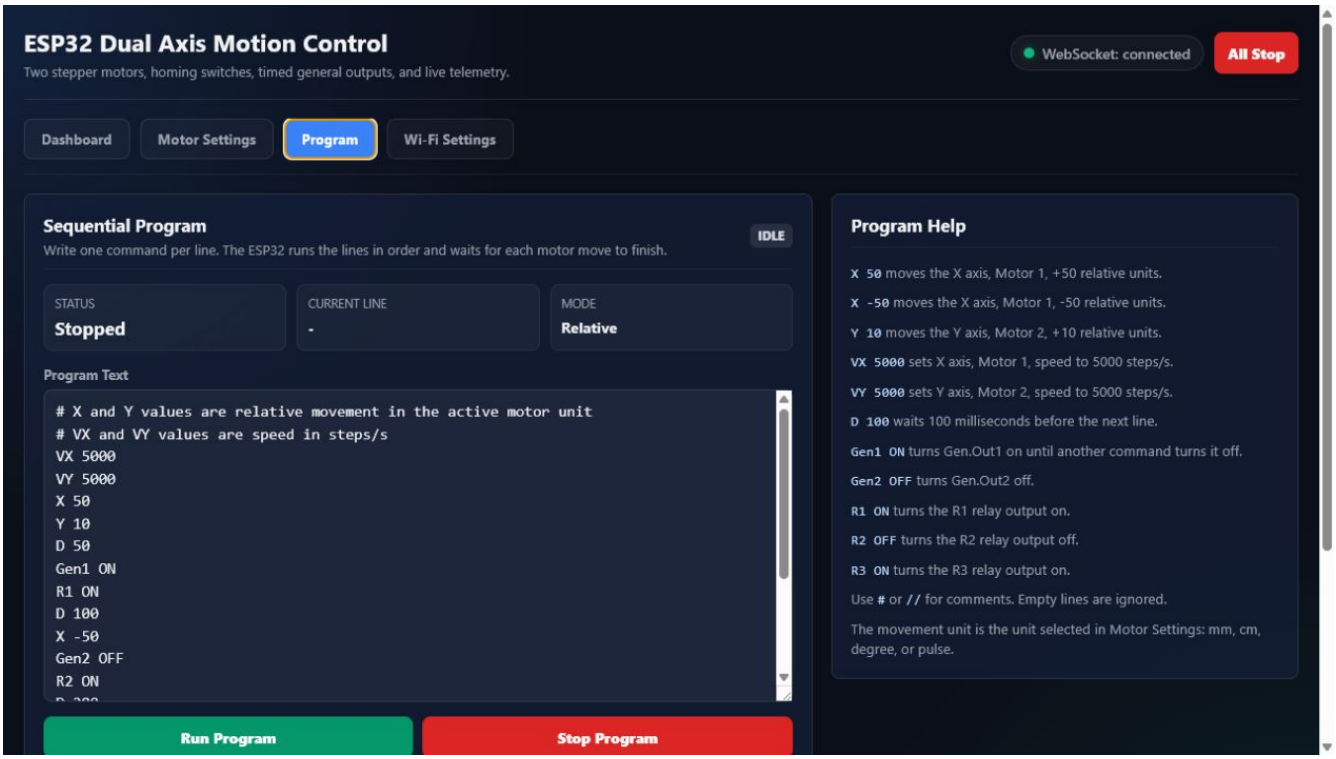
Dashboard tab: motor telemetry, manual axis control, and timed Gen.Out1 / Gen.Out2 outputs.



Motor Settings tab: speed, acceleration, limits, microstep, and mechanical unit settings.



Program tab: sequential text commands, Gen1 / Gen2 commands, and R1 / R2 / R3 output commands.



7. Program Tab

The Program tab runs text commands line by line. The ESP32 receives the program text and executes each line in order. When a motor move is started, the firmware waits for that motor move to finish before advancing to the next line. Delay commands do not block the main loop; WebSocket telemetry and watchdog service continue to run.

Example program:

```
VX 5000
VY 5000
X 50
Y 10
D 50
Gen1 ON
R1 ON
D 100
X -50
Gen2 OFF
R2 ON
D 200
R1 OFF
R2 OFF
R3 ON
D 100
R3 OFF
Gen1 OFF
```

In this example, the runtime speeds for the X and Y axes are set to 5000 steps/s, X moves +50 relative units, Y moves +10 relative units, the program waits 50 ms, Gen.Out1 and R1 turn on, the program waits 100 ms, X moves -50 relative units, Gen.Out2 turns off, R2 turns on, the program waits 200 ms, R1 and R2 turn off, R3 turns on briefly and then turns off, and Gen.Out1 turns off.

Command	Description
X <value>	Relative move for X axis / Motor 1. The value uses the active motor interface unit.
Y <value>	Relative move for Y axis / Motor 2. The value uses the active motor interface unit.
VX <steps/s>	Changes the X axis runtime speed in steps/s. This does not save to NVS.
VY <steps/s>	Changes the Y axis runtime speed in steps/s. This does not save to NVS.
D <ms>	Waits for the specified number of milliseconds. Example: D 100
WAIT <ms>	Readable alias for the D command.
Gen1 ON / OFF	Turns the Gen.Out1 output on or off.
Gen2 ON / OFF	Turns the Gen.Out2 output on or off.
R1 ON / OFF	Turns the R1 program relay output on GPIO2 on or off.
R2 ON / OFF	Turns the R2 program relay output on GPIO12 on or off.
R3 ON / OFF	Turns the R3 program relay output on GPIO15 on or off.
# or //	Starts a comment. Empty lines are ignored.

VX and VY speed values are accepted from 1 to 200000 steps/s. An invalid command, invalid number, or soft limit violation stops the program in an error state and writes the reason to the log panel.

Use Gen1 / Gen2 for the Gen.Out outputs. R1 / R2 / R3 control the separate program relay outputs.

8. Motor Settings Tab

The Motor Settings tab configures speed limit, acceleration, soft limits, step angle, microstep mode, mechanical drive type, and interface unit. The Save And Apply Settings button saves the values to NVS and applies them to the motors.

Program X and Y movement values are interpreted using the unit selected in this tab: pulse, mm, cm, or degree. VX and VY are always expressed in steps/s.

9. Wi-Fi Settings Tab

The Wi-Fi Settings tab saves SSID and password values so the ESP32 can join a local network. A restart is required after saving. AP mode always remains available, so the controller can still be reached through the ESP32-MOTOR access point even when router connection is unavailable.

10. Build And Upload

Arduino IDE or Arduino CLI can be used. For typical ESP32 DevKit boards, select the `esp32:esp32:esp32` board profile, also known as ESP32 Dev Module.

Required libraries: FastAccelStepper, ESP Async WebServer, Async TCP, and ArduinoJson. Before uploading, confirm the pin assignments in `main/Config.h`.

11. Safety Notes

- Set the DRV8825 current limit before running the motors.
- Run initial tests with low speed and low acceleration.
- If Gen.Out or R1/R2/R3 outputs control high-current or AC loads, use a suitable relay module, fusing, isolation, and enclosure.
- Firmware-based All Stop is not a complete machine safety system. Add a hardware emergency stop where required.
- Test home switch wiring before starting mechanical motion.